

SNAKE GAME

1. Project Overview

This project implements the classic **Snake Game** using two very different technological approaches:

1. **x86 Assembly (DOS / Real Mode)**: a low-level implementation that directly interacts with BIOS and DOS interrupts.
2. **Arduino (Embedded System)**: a modern implementation using a microcontroller, TFT LCD display, joystick input, LEDs, and a buzzer.

The goal of the project is not only to create a playable game, but also to **compare low-level system programming with modern embedded development**, highlighting how the same logical game can be expressed across radically different hardware and software environments.

2. Project Scope from the Assembly Perspective

2.1 Game Representation

- The screen is treated as a **character grid**.
- The snake is represented as a list of screen coordinates stored in memory.
- Each snake segment stores its **X and Y coordinates packed into a single 16-bit word**.
- The snake moves by manually shifting memory contents.

This approach emphasizes:

- Manual memory management
- Direct control over execution flow

2.2 Input Handling

- Keyboard input is read using **BIOS interrupt INT 16h**.
- Direction control uses **keyboard scan codes** rather than ASCII characters.
- The ESC key (ASCII 27) is used to exit the game.

This reflects how early PC games handled input at the hardware level.

2.3 Timing and Game Loop

- Timing is handled using **BIOS timer ticks** via INT 1Ah.
 - A busy-wait loop is used to control the game speed.
 - The game loop runs continuously without multitasking or OS scheduling.
-

3. Project Scope from the Arduino Perspective

3.1 Environment and Platform

The Arduino version runs on a **microcontroller-based embedded system**, using:

- Arduino-compatible board
- **TFT LCD display** for graphics
- **Analog joystick** for input
- LEDs and buzzer for feedback
- Arduino libraries for hardware abstraction

3.2 Game Representation

- The game is rendered in **pixel-based graphics** on a 480×320 TFT screen.
- The snake is represented using two arrays:
 - snakeX[] for X positions
 - snakeY[] for Y positions
- Each segment occupies a fixed-size grid cell.

This design improves readability and scalability compared to packed memory representations.

3.3 Input Handling

- A **joystick module** is used instead of a keyboard.
- Direction is determined by reading:
 - Analog values (X/Y axes)
 - A digital push button

This allows smooth and intuitive control suitable for embedded systems.

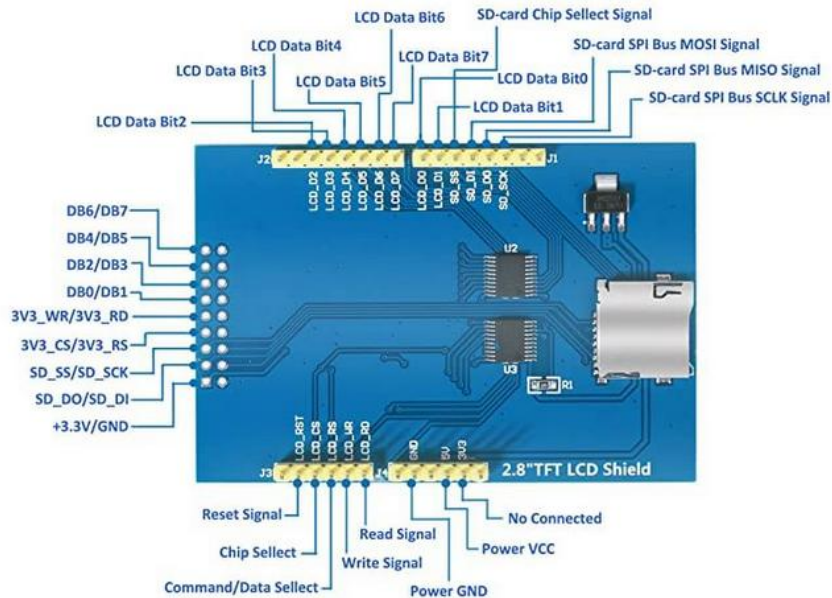
3.4 Timing and Game Loop

- Timing is handled using the millis() function.
- The game loop is **non-blocking**, allowing:
 - Smooth animation
 - LED updates
 - Sound playback

This reflects modern embedded programming practices.

4. Hardware Datasheets (Summary)

4.1 TFT LCD Display 3.5 inch Datasheet:

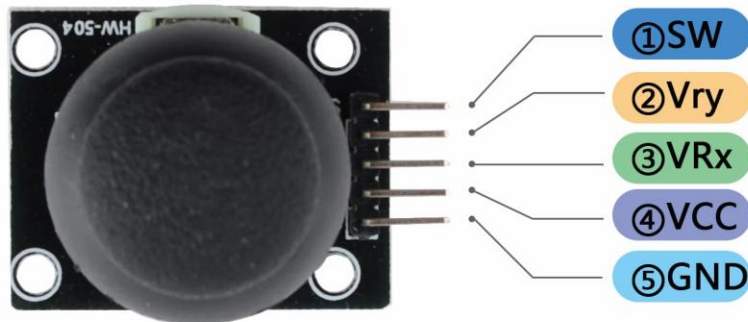


Pin definition

Pin No.	Symbol	Description
1	GND	Ground
2	VDD	Power supply (TYP=2.8V)
3	VDD	Power supply (TYP=2.8V)
4	CS	Chip select signal
5	RS	Register select signal
6	WR	Write strobe signal
7	RD	Read strobe signal
8	RST	Reset pin
9-24	DB0~DB15	Date Bus
25	GND	Ground
26	YD	Touch Pangel Input
27	XR	
28	YU	
29	XL	
30	LEDK1	Cathode of Back Light
31	LEDK2	
32	LEDK3	
33	LEDK4	
34	LEDK5	
35-36	LEDA	Anode of Back Light
37	GND	Ground

4.2 Joystick Datasheet:

Joystick Pinout



NO.	PIN	Function
1	SW	SW is the output from the pushbutton. It's normally open, meaning the digital readout from the SW pin will be HIGH. When the button is pushed, it will connect to GND, giving output LOW.
2	Vry	VRy gives readout of the joystick in the vertical direction (Y-coordinate) i.e. how far up and down the joystick is pushed.
3	VRx	VRx gives readout of the joystick in the horizontal direction (X-coordinate) i.e. how far left and right the joystick is pushed.
4	VCC	VCC supplies power for the module. You can connect it to 5V output from your Arduino.
5	GND	GND is the Ground Pin which we connect the GND pin on the Arduino.